# 스케줄링 알고리즘을 활용한 다중 사용자 LLM API 요청 관리 시스템

*Multi-User LLM API Request Management System Using Scheduling Algorithms*


| | |
|---|---|
| **발표자** | 서민지 |
| **소속** | 홍익대학교 컴퓨터공학과 4학년 |
| **지도교수** | 이장호 교수님 |
| **발표일** | 2026년 5월 |

# 목차


1 **연구 배경 및 목적**
문제 정의 · 연구 질문

2 **시스템 설계**
아키텍처 · 4계층 구조

3 **구현**
4가지 스케줄링 알고리즘 · 등급 체계

4 **실험 결과**
성능 비교 · MLFQ 선점형 · WFQ 차등 서비스

5 **결론**
연구 요약 · 한계 · 향후 과제

# 연구 배경 및 목적

## 문제 상황

- LLM API 서비스 다중 사용자 동시 접속 급증
- 요청 처리 순서· 우선순위 기준 없음
- 특정 사용자 독점 시 다른 사용자 대기 지연
- 구독 등급별 차등 서비스 불가

→

## 제안 접근법

**OS 스케줄링 알고리즘을**
**LLM API 요청 관리에 적용**

FCFS · Priority · MLFQ · WFQ
4가지 알고리즘 구현 및 비교

## 연구 질문 (Research Questions)

| | |
|---|---|
| RQ1 | OS 스케줄링 알고리즘을 LLM API 환경에 구현할 수 있는가? |
| RQ2 | 알고리즘별 성능(대기시간)은 어떻게 다른가? |
| RQ3 | 사용자 공정성(JFI)은 알고리즘별로 어떻게 차이나는가? |

# 시스템 아키텍처

**클라이언트 계층**
REST Client · 대시보드

↓

**API 계층**
Express.js · Rate Limiter(전처리) · 요청 접수

↓

**스케줄러 계층**
FCFS · Priority · MLFQ · WFQ

↓

**저장소 / LLM 계층**
메모리 큐 · JSON 파일 로그 · Ollama

### 기술 스택

- Node.js 22
- Express.js
- JavaScript
- 메모리 큐 + JSON 로그
- Ollama (LLM)

# 스케줄링 알고리즘 비교

## FCFS

First-Come, First-Served

**도착 순서대로 처리**

핵심: 단순, 공평, 순서 보장

## Priority

Priority Scheduling

**긴급한 요청 먼저 처리**

핵심: URGENT > HIGH > NORMAL > LOW

## MLFQ

Multi-Level Feedback Queue

**짧은 요청 우선 처리**

핵심: 큐 수준에 따른 시간 할당량

## WFQ

Weighted Fair Queuing

**등급별 차등 서비스**

핵심: Enterprise > Premium > Standard > Free

# 등급 체계 설계

## 요청 우선순위 (Request Priority)

*Priority Scheduler 사용*

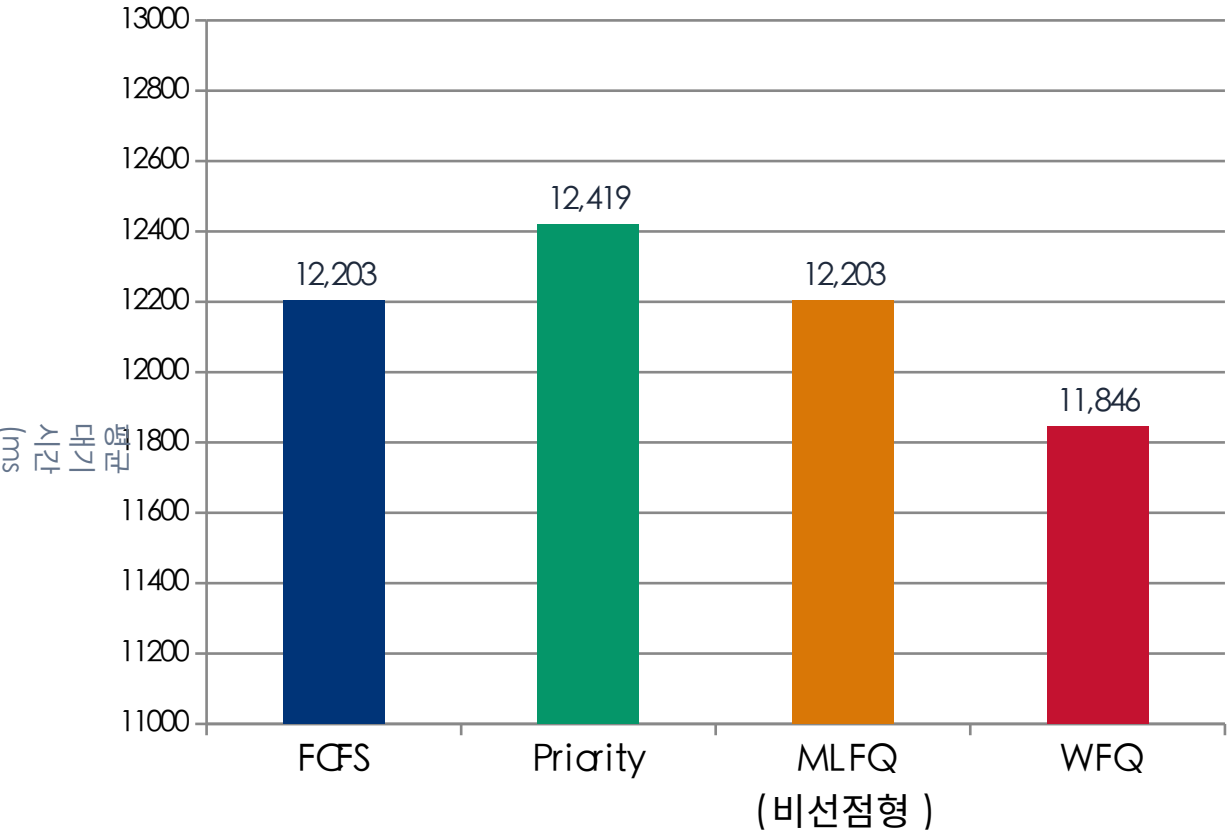
| | |
|---|---|
| URGENT | 즉시 처리 필요 |
| HIGH | 우선 처리 |
| NORMAL | 일반 처리 |
| LOW | 여유 처리 |

## 구독 등급 (Subscription Tier)

*WFQ Scheduler · Rate Limiter 사용*

| | |
|---|---|
| Enterprise | 가중치 100<br>분당 100건 |
| Premium | 가중치 50<br>분당 50건 |
| Standard | 가중치 10<br>분당 10건 |
| Free | 가중치 1<br>분당 5건 |

# 실험 결과 1: 알고리즘별 평균 대기시간




* 시뮬레이션 500건, 처리시간 10~100ms 균등 분포

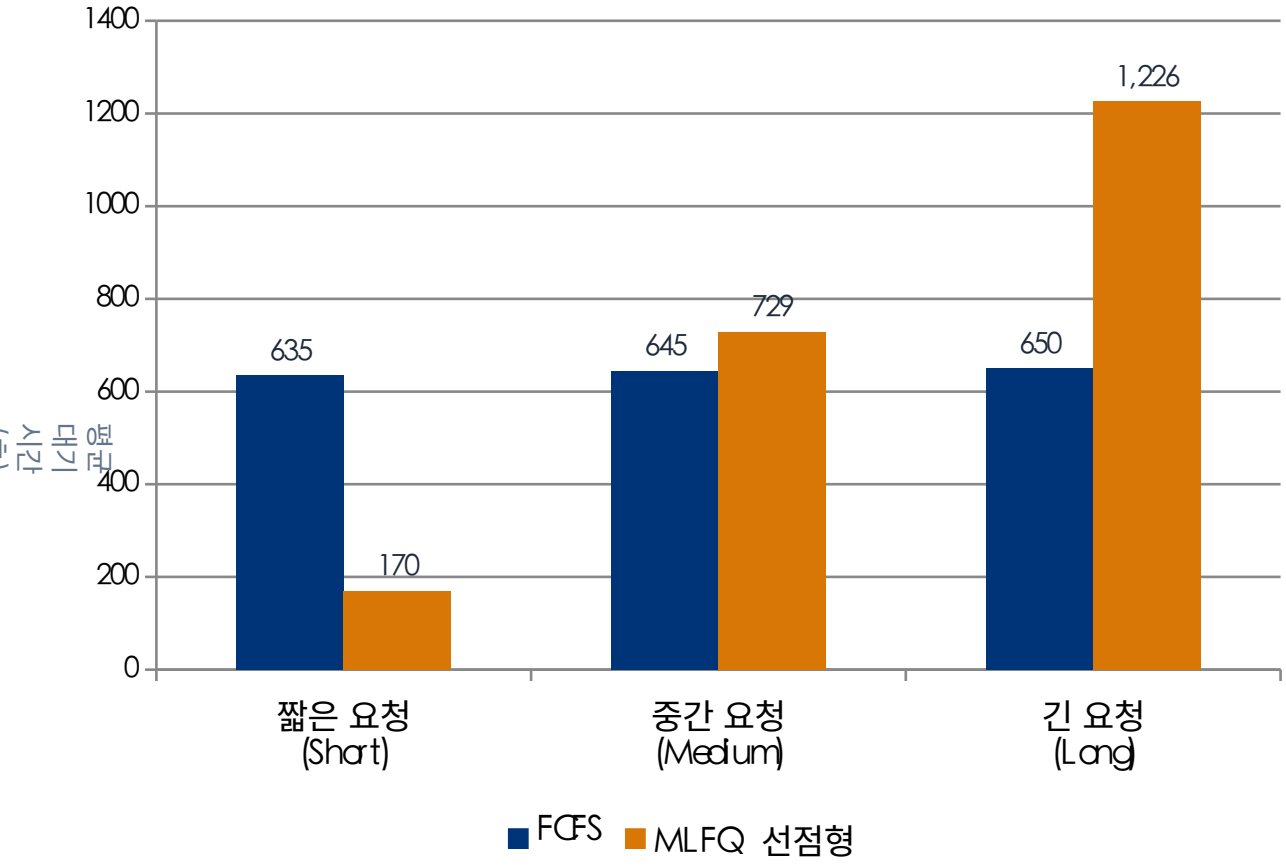
## 핵심 발견

! MLFQ(비선점형) = FCFS 동일 결과

✓ WFQ가 가장 짧은 평균 대기시간

→ 선점 없이는 큐 순서와 무관

# 실험 결과 2: MLFQ 선점형 vs FCFS




**짧은 요청 대기시간**

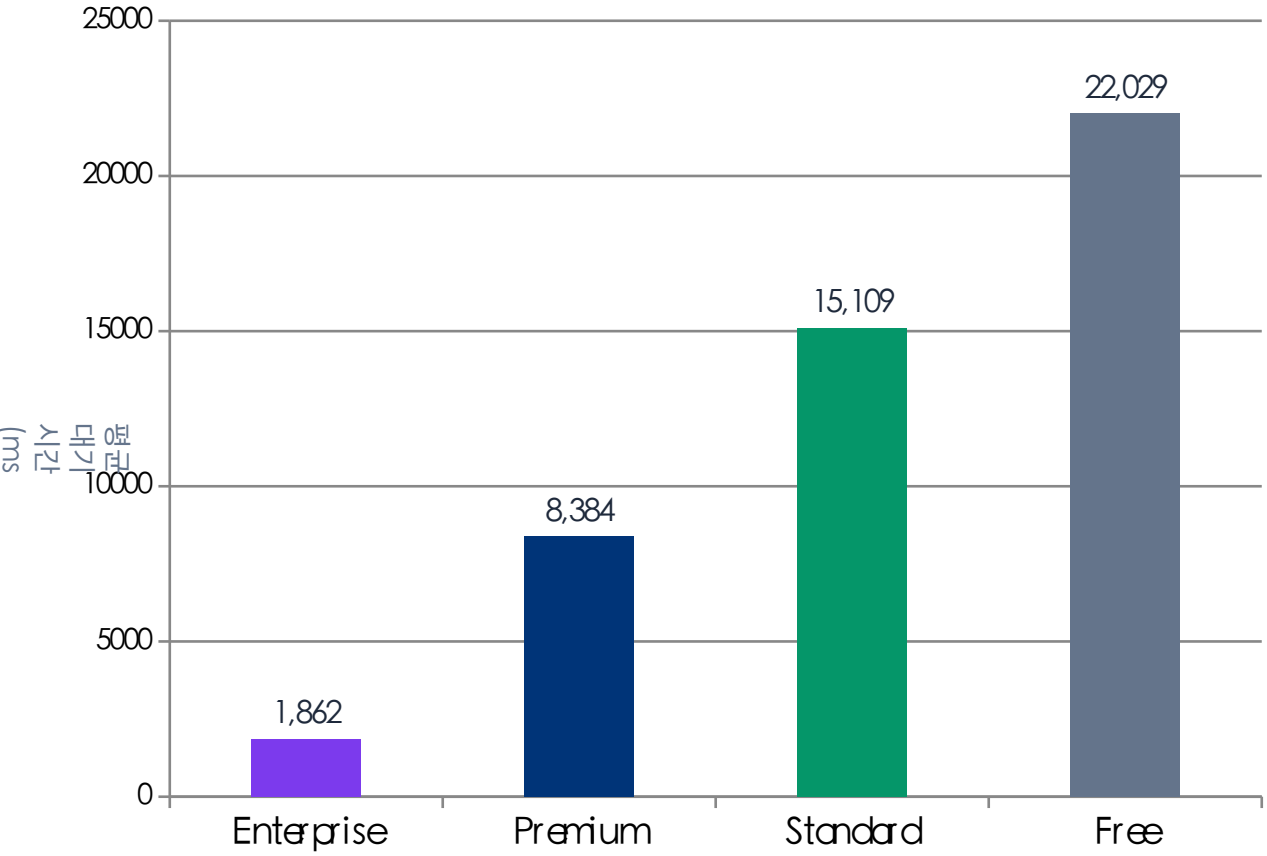
**약 73% 감소**

635초 → 170초

**트레이드오프**

| | |
|---|---|
| Short | ↓73% |
| Medium | ↑13% |
| Long | ↑89% |

# 실험 결과 3: WFQ 구독 등급별 대기시간




## 핵심 결과

**Enterprise**

**약 1,862ms**

VS

**Free**

**약 22,029ms**

**약 12배 차이**

가중치 비율에 비례한
차등 서비스 확인

# 실험 결과 4: 실서버(Ollama) 검증

## 시뮬레이션

- 처리시간: 10~100ms (미리 설정)
- 스케줄러: 처리 순서 결정
- 결과: 즉시 반환
- 선점형: 구현 가능
- 장점: 빠른 반복 실험

## 실서버 (Ollama LLM)

- 처리시간: 약 208ms (실제 추론)
- 스케줄러: 처리 순서 결정
- 결과: 실제 LLM 응답 생성
- 선점형: 불가 (추론 중단 불가)
- 장점: 실제 환경 검증

핵심 발견: 스케줄러는 처리 순서만 결정하며, 실제 LLM 추론 속도에는 영향을 주지 않습니다.
실서버 환경에서도 스케줄링 알고리즘이 정상 동작하였고, 선점형 구현은 실제 LLM에서 제약이 있습니다.

# 결론

**RQ1**

### 구현 가능성

4개 알고리즘 모두 구현 완료
Ollama 실서버 연동까지 검증

**RQ2**

### 성능 비교

비선점형 환경에서 알고리즘 간 처리량은 유사
WFQ 평균 대기시간 최소, MLFQ 선점형은 짧은 요청 약 73% 감소

**RQ3**

### 공정성 분석

FCFS· Priority· MLFQ: JFI = 1.0 (완전 공평)
WFQ: JFI = 0.32 (의도적 차등)

### 한계 및 향후 과제

- 시뮬레이션 기반 실험 (실제 부하와 차이 가능)
- 선점형은 실제 LLM에서 적용 불가
- 향후: 토큰 기반 비용 모델 · 대시보드 고도화 · 등급 내 공정성 검증 · 대규모 실험

# 실시간 데모

**http://localhost:3000**

### 시나리오 1: FCFS vs WFQ 비교 (2분 30초)

① FCFS 서버 시작

② 4개 등급 요청 전송

③ 처리 순서 확인 (도착 순서)

④ WFQ로 전환 후 재전송

⑤ Enterprise 우선 처리 확인

### 시나리오 2: Rate Limiter 시연 (1분)

① Rate Limiter 활성화

② Free 등급 6개 요청 전송

③ 429 응답 확인 (한도 초과)

④ Enterprise는 정상 처리 확인

```
SCHEDULER_TYPE=WFQ node src-simple/server.js
curl -X POST http://localhost:3000/api/requests -H "Content-Type: application/json" \
  -d '{"prompt":"Hello","tenantId":"enterprise","tier":"enterprise","priority":"HIGH"}'
```